Engineering Design Document for Django GreatCart Project (System Memory)

Date Created: 2026-06-09**Date Last Updated:** 2026-06-29**Author:** Albasha

1. System Overview

This document aims to provide a professional and comprehensive engineering reference for the Django GreatCart project, focusing on the current architecture, core business rules, data relationships, and design decisions. The project was developed using a Modular Apps philosophy to ensure flexibility and scalability.

1.1 Core Applications

App	Description
`accounts`	User management and Custom User Model authentication system.
`category`	Product category management.
`store`	Management of products, search, and variations.
`carts`	Shopping cart management, add/remove products, and quantity logic.
`orders`	Order management, order details, and payment processing.

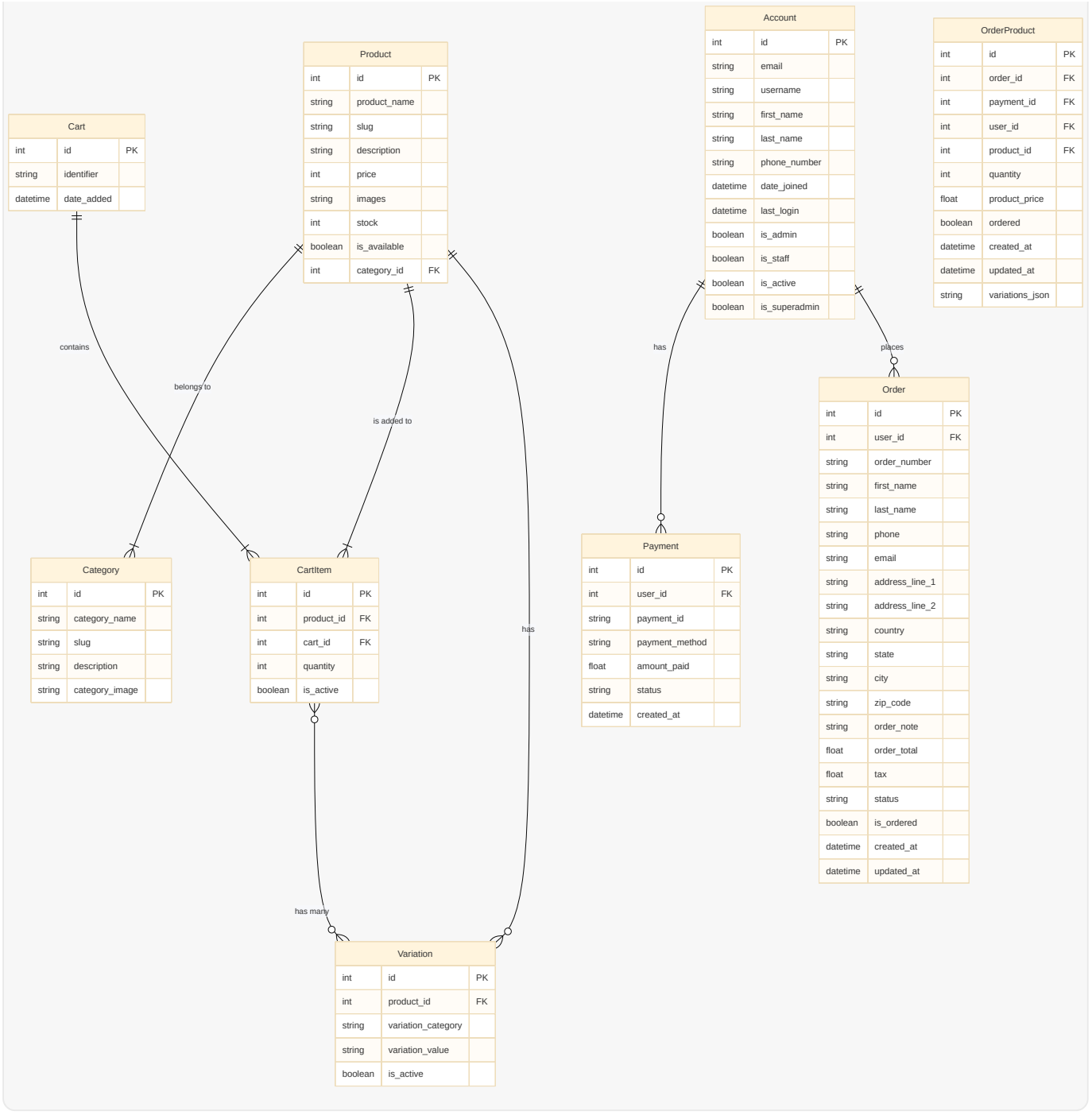
---

2. Models Layer & Data Relationships

This layer illustrates the database structure and relationships between different models, focusing on field names and types.

2.1 Data Relationships Map

```mermaid
```



2.2 Model Details

2.2.1 `accounts.Account` (Custom User Model)

- **Base:** `AbstractBaseUser` (User model built from scratch).
- **Fields:**
  - `email` : `EmailField(max_length=100, unique=True)` - Primary identifier for login.
  - `username` : `CharField(max_length=50, unique=True)` .
  - `first_name` , `last_name` : `CharField(max_length=50)` .

    - `phone_number`: `CharField(max_length=50)`.
    - `date_joined`, `last_login`: `DateTimeField(auto_now_add=True/True)`.
    - `is_admin`, `is_staff`, `is_active`, `is_superadmin`: `BooleanField(default=False)`.
- **Custom Manager (`MyAccountManager`):**
    - Inherits from `BaseUserManager`.
    - Contains `create_user` and `create_superuser` functions.
    - **Critical Point:** When saving a user within the manager, `user.save(using=self._db)` must be used to avoid `AttributeError`, as `_db` is the correct property to access the default database.
- **`settings.py` Configuration:** `AUTH_USER_MODEL = 'accounts.Account'`.
- **Authentication Updates:** `USERNAME_FIELD = 'email'` and `REQUIRED_FIELDS = ['username', 'first_name', 'last_name']` were specified, confirming that email is the primary login identifier.
- **New User Status:** `is_active = False` by default for new users, requiring email activation.

2.2.2 `category.Category`

- **Base:** `models.Model`.
- **Fields:**
    - `category_name`: `CharField(max_length=50, unique=True)`.
    - `slug`: `CharField(max_length=100, unique=True)` - Used for building clean (SEO-friendly) URLs.
    - `description`: `TextField(blank=True)`.
    - `category_image`: `ImageField(upload_to='photos/categories', blank=True)`.
- **`Meta` Class:**
    - `verbose_name = 'category'`.
    - `verbose_name_plural = 'categories'` (for better display in the admin panel).

2.2.3 `store.Product`

- **Base:** `models.Model`.
- **Fields:**
    - `product_name`: `CharField(max_length=200, unique=True)`.
    - `slug`: `CharField(max_length=200, unique=True)`.
    - `description`: `TextField(blank=True)`.

- `price` : `IntegerField()` .
- `images` : `ImageField(upload_to='photos/products')` .
- `stock` : `IntegerField()` - Available product quantity.
- `is_available` : `BooleanField(default=True)` - Determines if the product is available for purchase.
- `category` : `ForeignKey(Category, on_delete=models.CASCADE)` .

2.2.4 `store.Variation`

- **Base:** `models.Model` .
- **Fields:**
  - `product` : `ForeignKey(Product, on_delete=models.CASCADE)` .
  - `variation_category` : `CharField(max_length=100, choices=[('color', 'color'), ('size', 'size')])` .
  - `variation_value` : `CharField(max_length=100)` .
  - `is_active` : `BooleanField(default=True)` .
- **Custom Manager (** `VariationManager` **):**
  - Contains functions like `colors()` and `sizes()` to filter active options based on category.

2.2.5 `carts.Cart`

- **Base:** `models.Model` .
- **Fields:**
  - `identifier` : `CharField(max_length=250, unique=True)` - Stores `session_key` for guests or `user.id` for registered users.
  - `date_added` : `DateField(auto_now_add=True)` .

2.2.6 `carts.CartItem`

- **Base:** `models.Model` .
- **Fields:**
  - `product` : `ForeignKey(Product, on_delete=models.CASCADE)` .
  - `variations` : `ManyToManyField(Variation, blank=True)` - Allows linking multiple options (color, size) to the same cart item.
  - `cart` : `ForeignKey(Cart, on_delete=models.CASCADE)` .
  - `quantity` : `IntegerField()` .

- `is_active` : `BooleanField(default=True)` .

2.2.7 `orders.Order`

- **Base:** `models.Model` .
- **Fields:**
  - `user` : `ForeignKey(Account, on_delete=models.SET_NULL, null=True)` - The user who placed the order.
  - `payment` : `ForeignKey(Payment, on_delete=models.SET_NULL, blank=True, null=True)` - Payment transaction linked to the order.
  - `order_number` : `CharField(max_length=20, unique=True)` - Unique order number.
  - `first_name` , `last_name` , `phone` , `email` , `address_line_1` , `address_line_2` , `country` , `state` , `city` , `zip_code` , `order_note` : Shipping detail fields.
  - `order_total` , `tax` : `FloatField()` - Total order amount and tax.
  - `status` : `CharField(max_length=10, choices=[('New', 'New'), ('Accepted', 'Accepted'), ('Completed', 'Completed'), ('Cancelled', 'Cancelled')], default='New')` .
  - `is_ordered` : `BooleanField(default=False)` - Determines if the order has been successfully completed.
  - `created_at` , `updated_at` : `DateTimeField(auto_now_add=True/True)` .

2.2.8 `orders.Payment`

- **Base:** `models.Model` .
- **Fields:**
  - `user` : `ForeignKey(Account, on_delete=models.CASCADE)` .
  - `payment_id` : `CharField(max_length=100)` - Transaction ID from the payment gateway.
  - `payment_method` : `CharField(max_length=100)` .
  - `amount_paid` : `CharField(max_length=100)` - Amount paid.
  - `status` : `CharField(max_length=100)` - Payment status (e.g., Completed, Failed).
  - `created_at` : `DateTimeField(auto_now_add=True)` .

2.2.9 `orders.OrderProduct`

- **Base:** `models.Model` .
- **Fields:**
  - `order` : `ForeignKey(Order, on_delete=models.CASCADE)` .

- `payment` : `ForeignKey(Payment, on_delete=models.SET_NULL, blank=True, null=True)` .
- `user` : `ForeignKey(Account, on_delete=models.CASCADE)` .
- `product` : `ForeignKey(Product, on_delete=models.CASCADE)` .
- `variations` : `ManyToManyField(Variation, blank=True)` - Variations linked to the product at the time of purchase.
- `quantity` : `IntegerField()` .
- `product_price` : `FloatField()` - Product price at the time of purchase.
- `ordered` : `BooleanField(default=False)` .
- `created_at` , `updated_at` : `DateTimeField(auto_now_add=True/True)` .
- `variations_json` : `JSONField(blank=True, null=True)` - To store variations as a dictionary list for consistency.

---

3. Business Logic Layer & Core Business Rules

This layer describes the core rules and processes governing system behavior.

3.1 Core Business Rules

These are the system "laws" that must be respected in all operations:

- **CartItem Uniqueness Rule:** A unique `CartItem` in the cart is defined by **Product + Set of Variations**. Any change in variations (even for the same product) results in a new `CartItem` .
- **Stock Management Rule:**
  - Product `stock` is **decreased** when a new `CartItem` is added or its quantity is increased in the cart.
  - Product `stock` is **increased** when a `CartItem` is deleted or its quantity is decreased in the cart.
  - Product `stock` must be **greater than zero** (`stock > 0`) and `is_available = True` to allow adding the product to the cart.
  - Product `stock` is permanently **decreased** upon successful payment completion.
- **Session/User-Based Cart Rule:** Each user session (Guest) or logged-in user has an independent shopping cart. The session cart is merged with the user cart upon login.
- **Variations Rule:** Variations (such as color and size) define behavior similar to an `SKU` (Stock Keeping Unit) for a single product, rather than separate products.

- **Pending Orders Rule:** Prevents creating a new order if the user has a previous order that is not yet completed.

3.2 Add to Cart Logic (`add_cart`)

This is the most complex part of the project, relying on:

1. **Receiving Variations:** `color` and `size` are fetched from the `POST` request.
2. **Stock Verification:** `product.stock > 0` and `product.is_available` are checked in the `add_cart` function before any addition to ensure no unavailable products are added or stock limits exceeded. This represents a **Backend security check**.
3. **Variation Matching Algorithm:**

- When adding a product with variations, a list of `Variation` objects is created from the `POST` request.
- The cart is searched for a `CartItem` matching the current product with the exact same set of variations.
- **How comparison is done:** The list of `Variation` objects for each existing `CartItem` in the cart is converted into a list of lists (or an ordered set) to compare with the new variation list. This ensures order matters in matching.
  - **Result:**
    - If a matching `CartItem` is found (same product and same variations), the `quantity` for this `CartItem` increases.
    - If no matching `CartItem` is found (same product but different variations), a new `CartItem` is created.

3.3 Session Logic (`_cart_id`) and Identifier Unification

- **Applied Behavior:** The `_cart_id(request)` function retrieves the current `session_key` for guests or `user.id` for registered users. If no `session_key` exists, a new session is created.
- **Logical Fix:** The previous logical error was corrected to ensure a new session is created.
- **Identifier Unification:** Using a single identifier simplifies cart management across guest and authenticated states.

3.4 Cart Merge Logic

Intelligent logic was developed to handle the shopping cart when a user transitions from "Guest" to "Authenticated User":

- **Merge Algorithm in `login_page` :** Upon login, the system:
  1. Checks for a cart associated with the `session_key` .

2. If it exists, it searches for a cart associated with the user (`user.id`).
3. **Merge Items:** Items are moved from the session cart to the user cart while respecting "Item Uniqueness" (Product + Variations).- If the product with the same variations already exists in the user cart, only the quantity is increased.- If the product has different data or does not exist, a new `CartItem` linked to the user cart is created.
4. **Cleanup:** The old session cart is deleted after merging to ensure database cleanliness.

3.5 Search Logic

- Django `Q objects` are used for comprehensive searching across multiple fields (e.g., `product_name` , `description` , `category__category_name`) to provide accurate search results.

---

4. Authentication & Security System

This chapter documents recent developments in the Django GreatCart project related to user management, account activation, login/logout, and the password reset system. These components form the security backbone of the project.

4.1 Email Account Verification System

An account activation system was implemented to ensure the validity of user email addresses and prevent fake account creation. The system relies on the following steps:

- **Core Rule:** When a new user registers, `is_active = False` is set in their `Account` model. The user cannot log in until their account is activated.
- **Sending Process (`register` View):**
  - After saving the `RegisterForm` , `get_current_site(request)` is called to get the current domain name.
    - An encoded `uid` (user identifier) is generated using `urlsafe_base64_encode(force_bytes(user.pk))` .
      - **Engineering Note:** `uidb64` is merely an `Encoding` (data format transformation), not `Encryption` . It can be easily decoded and provides no security on its own; rather, it ensures the identifier is safe for URL use (URL-safe).
  - A security `token` is generated using `default_token_generator.make_token(user)` . This `token` is the real security component, as it depends on user data (like `user.pk` and

`password_hash`) and automatically expires when the link is used or the password changes.

- The email message is built using the `account_verification_email.html` template via `render_to_string`, receiving `user`, `domain`, `uid`, and `token`.
- The email is sent using `EmailMessage` with SMTP settings specified in `settings.py`.
- After sending, the user is redirected to the login page with `Query Parameters` (`command=verification&email=...`) to display a guidance message.

- **Activation Process (`activate` View):**
  - The function receives `uidb64` and `token` from the link.
  - `uidb64` is decoded using `urlsafe_base64_decode().decode()` to recover `user.pk`.
  - The user is fetched from the database using `Account._default_manager.get(pk=uid)`.
  - The `token` validity is checked using `default_token_generator.check_token(user, token)`.
  - If the `token` is valid, `user.is_active = True` is set, the user is saved, and then redirected to the login page with a success message.

4.2 Login & Logout System

A secure and efficient login/logout system was built using built-in Django tools:

- **Login Page (`login_page` View):**
  - First checks if the user is already authenticated (`request.user.is_authenticated`), in which case they are redirected to the `home` page to avoid showing the login page to an already logged-in user.
  - Receives email and password data from the `POST` form.
  - **Email Processing:** The email is cleaned (`.strip().lower()`) to ensure consistency in comparison.
    - **User Existence Check:** `try-except Account.DoesNotExist` is used to distinguish between "User does not exist" and "Incorrect password," providing more accurate error messages.
      - **Authentication (`authenticate`):** The `authenticate(request, email=email, password=password)` function is used to verify credentials. It returns a `User` object if correct, or `None` if incorrect.
        - **Note:** Since `USERNAME_FIELD = 'email'` in the `Account` model, `authenticate` uses email as the primary identifier.
- **Login (`login`):** If authentication succeeds, the `login(request, user)` function creates a `Session` for the user, making them logged in for subsequent requests (`request.user` becomes the user object). Cart merge logic is also called here.

    - **Smart Redirection:** `return redirect(request.GET.get('next', 'home'))` allows redirecting the user back to the page they were trying to access before logging in (via `?next=/path/`), or to the home page by default.
- **Logout Page (`logout_page` View):**
    - Uses the `logout(request)` function to delete the user session, ending their login. It does not delete the user from the database.
    - Redirects the user to the home page.

4.3 Password Reset System

The password reset system was implemented in two phases: first manually for learning purposes, and second using official Django Class-Based Views (CBVs) for security and efficiency.

- **Conceptual Idea:** The old password is not retrieved (as it is hashed); instead, a new password is created.
- **Protection Against User Enumeration:** At all stages of password reset, the system displays a generic message to the user (e.g., "If this email is registered, we have sent recovery instructions"), regardless of whether the email exists in the database. This prevents attackers from identifying valid accounts.
- **Manual Phase (FBVs - for learning purposes):**
    - `password_reset`, `password_reset_done`, and `password_change` functions were built manually.
    - Used the same `uidb64` and `token` principles to generate secure, temporary links.
    - **Security Point:** `user.set_password(new_password)` was used to hash the new password before saving, which is crucial.
- **Official Phase (CBVs - Current Status):**
    - Manual functions were replaced with the following CBVs from `django.contrib.auth.views`:
        - `PasswordResetView`: Handles displaying the email request form, generating `uidb64` and `token`, and automatically sending the email using `settings.py` configurations.
            - `PasswordResetDoneView`: Displays an email sent confirmation message.
                - `CustomPasswordResetConfirmView`: Customized from `PasswordResetConfirmView` to:
                    - Add a success message using `messages.success` after a successful password change.

- Specify `success_url = reverse_lazy('login')` to redirect the user directly to the login page after updating the password, instead of the default `PasswordResetCompleteView` page.

4.4 Messages Framework

- **Goal:** Provide a unified mechanism to display feedback messages to the user (success, error, warning) via the user interface.
- **Mechanism:**
  - `messages.success()`, `messages.error()`, etc., are called in the `views`.
  - Django temporarily stores these messages in the user's `Session`.
  - Messages are automatically injected into HTML templates via `django.contrib.messages.context_processors.messages`.
  - After displaying the message in the template (usually via `templates/includes/alerts.html` included in `base.html`), it is automatically deleted from the session (Flash Messages).
  - **Customization:** `MESSAGE_TAGS` was configured in `settings.py` to map Django message levels to appropriate Bootstrap CSS classes (`danger`, `success`, `warning`, `info`).

4.5 User Management System

An integrated dashboard was built to allow users to manage their activity within the store:

- **Dashboard:** Displays a quick summary for the user (e.g., number of orders, general status) and serves as a gateway to all personal functions.
- **My Orders:**
  - Displays all orders placed by the user with their details (order number, date, total, status).
  - Data is fetched using smart filtering: `Order.objects.filter(user=request.user, is_ordered=True)`.
- **Profile Settings:**
  - Allows users to update their personal information (name, phone, address).
  - Data is handled via custom forms ensuring input validity before saving.

4.6 Security Evolution (Password Change)

This part underwent a significant engineering journey reflecting project maturity:

- **Phase One (FBVs - Manual Method):** Started by building the `change_password` function manually using `PasswordChangeForm`. The goal was to understand "behind the scenes" how the old password is verified, how hashing works, and the necessity of using `update_session_auth_hash` to prevent the user from being logged out.
- **Phase Two (CBVs - Advanced Method):** After grasping the logic, transitioned to using Django's built-in `PasswordChangeView`.
  - **Engineering Reason:** Reduced code from 20 lines to a single line, ensured adherence to the highest official security standards, and simplified customization via Templates.

---

5. Order Lifecycle

The project transitioned from merely saving an order record to managing a full lifecycle consisting of the following stages:

A. Place Order

- `OrderForm` (a `ModelForm`) is used to collect shipping data from the user.
- Total and tax are calculated on the Backend to ensure accuracy and security.
- A unique `order_number` is generated based on the date and order ID (e.g., `2025081115`).
- **Innovation:** Products are moved from `CartItem` (temporary data) to `OrderProduct` (fixed historical data) as soon as the order is created, ensuring the product state and price at the time of purchase are preserved.

B. Payment & Review Page

- An intermediate page allowing the user to review shipping details and products before final payment.
- The page implements protection against "multiple pending orders," preventing a new order from being created if the user has a previous uncompleted order.

C. Fake Payment System

- Due to PayPal's geographical complexities, a "Demo Wallet" system was built using JavaScript and Fetch API.
- Payment data (transaction ID, status, method) is sent via a `POST` request in JSON format to the `payments` function.
- The function uses `json.loads(request.body)` to process the data and update the order status to `is_ordered = True` and `status = 'Completed'`.

D. Post-Payment Processing

Once payment is successful, the system automatically:

1. **Updates Stock:** Decreases the quantity of sold products from the store inventory (`product.stock -= quantity`).
2. **Confirms Products:** Updates the `OrderProduct` status to link them to the payment transaction and changes their status to `ordered = True` .
3. **Clears Cart:** Deletes all items from the user's shopping cart.
4. **Sends Email:** Sends an order confirmation email to the user with order details.
5. **Redirects:** Sends the user to the "Order Complete" page with the transaction ID and order number.

---

6. Configuration Layer

6.1 `settings.py` Core Settings

- **Custom User:** `AUTH_USER_MODEL = 'accounts.Account'` .
- **Media Files:** `MEDIA_URL` and `MEDIA_ROOT` were configured to manage uploading and displaying product and category images.
- **Context Processors:**
  - `django.template.context_processors.request`
  - `django.contrib.auth.context_processors.auth`
  - `django.contrib.messages.context_processors.messages`
  - `category.context_processors.menu_links` (to fetch categories for the menu).
  - `carts.context_processors.counter` (to count cart items).

---

7. Frontend & UI/UX Improvements

- **Unifying Card Heights:** The `fix-height` CSS class was used to ensure all product cards on the store page have the same height, improving the page's aesthetic appearance.

7.1 Review & Rating System

- **`ReviewRating` Model:** Links ratings to products and users, using `Avg` to automatically calculate the mean rating.

- **Security Rule:** Ratings can only be submitted after an actual purchase (`ordered=True`) to ensure credibility.
- **Display Algorithm:** Converts numerical ratings into percentages to visually represent stars in the interface.

7.2 Product Gallery

- Transformed product display from a "single static image" to an "interactive gallery."
- Used a `ForeignKey` relationship to link multiple images, with JavaScript logic to dynamically switch the main image without refreshing the page (Dynamic UI Update).

---

8. Design Decisions (The "Why")

This section explains the reasons behind some key design decisions:

- **Why `AbstractBaseUser` instead of `AbstractUser`?**
  - **Reason:** The need to build a fully custom user model from scratch, with complete control over all fields and behaviors, without being restricted by `AbstractUser` default fields. This provides greater flexibility for projects requiring a completely unique user structure.
- **Why Session-Based Cart?**
  - **Reason:** To provide a seamless shopping experience for unregistered visitors, allowing them to add products to the cart before logging in or creating an account. This reduces initial friction and increases conversion probability.
- **Why `ManyToManyField` for Variations in `CartItem` and `OrderProduct`?**
  - **Reason:** Flexibility in handling multiple sets of variations (e.g., color and size together) for the same product in a single cart item or order product. This ensures an accurate representation of the product chosen by the customer.
- **Why `JSONField` in `OrderProduct`?**
  - **Reason:** To ensure variation data consistency at the time of purchase. If the original product variations change later, the order record retains the variations actually purchased, maintaining the integrity of historical order data.

8.1 Final Design Decisions

Decision	Engineering Reason

Switching to CBVs for Password	To adopt officially tested secure code and reduce software maintenance.
Separating My Orders from Dashboard	To organize user experience and facilitate access to historical order data.
**Using `update_session_auth_hash`	To maintain user session continuity after changing sensitive credentials.
Linking Ratings to Actual Purchase	To protect the store's reputation and prevent random or malicious ratings.

---

9. Edge Cases & Known Risks

- **Stock Concurrency:** In high-load environments, concurrency issues (Race Conditions) may occur when updating stock. This can be mitigated using Database Locking mechanisms or Queues to ensure Atomic Updates.
- **Payment Failure:** The system must handle payment failures gracefully, providing retry mechanisms or clearly informing the user.
- `uidb64` **Security:** While URL-safe, it is not encrypted. Primary reliance must be placed on the security `token` to ensure the protection of activation and password reset links.

---

10. Project Status & Next Steps

Current Status: With these points completed, the project now possesses:

- A complete and secure **User System** (activation, recovery, change, profile).
- An integrated **Commerce System** (products, variations, cart, session merge, inventory).
- A robust **Order System** (creation, fake payment, invoices, email, order history).
- A reliable **Interaction System** (ratings, image gallery).

Future Enhancements:

- Integration of real payment gateways (e.g., Stripe or PayPal).
- Admin product management system (Admin Panel).
- UI/UX improvements.
- Adding advanced features like Wishlists and Product Comparisons.